

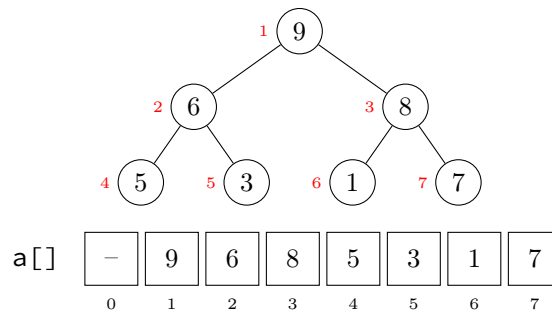
Lecture Review

Analysis of Algorithms

- *Experimental analysis.* Measure the running time $T(n)$ for inputs of increasing size n and plot $T(n)$ vs. n . Hypothesis: $T(n) = a \cdot n^b$ for constants a and b .
- *Doubling test.* Run the program on inputs whose size doubles each time. If $T(n) = a \cdot n^b$, then $b \approx \log_2 (T(n)/T(n/2))$; once b is known, solve for a using one $(n, T(n))$ pair.
- *Cost model & order of growth.* Pick a representative operation (e.g., array accesses) as a proxy for running time, then use tilde and Θ notation to discard lower-order terms and constants. Common orders of growth: $\Theta(1)$, $\Theta(\log n)$, $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^2)$, $\Theta(n^3)$, $\Theta(2^n)$.

Priority Queues

- *API.* A priority queue supports `insert(key)` and `delMax()` (or `delMin()`), returning/removing the largest (or smallest) key.
- *Elementary implementations.* Unordered list: `insert` in $\Theta(1)$, `delMax` in $\Theta(n)$. Ordered array: `insert` in $\Theta(n)$, `delMax` in $\Theta(1)$.
- *Binary heap.* Array representation (1-indexed) of a heap-ordered complete binary tree: the parent of the node at index k is at $k/2$, and its children are at $2k$ and $2k + 1$.



A heap-ordered complete binary tree and its array representation; index 0 is unused.

- *Swim and sink.* `insert` adds the new key at the end and *swims* it up to restore heap order; `delMax` exchanges the root with the last key, removes it, and *sinks* the new root down. Both run in $\Theta(\log n)$.

Lab Exercises

Part A: Empirical analysis of 3-SUM

Repeat the experiments from the slides to find the running time of brute-force 3Sum algorithm in the form an^b where a and b are constants. Run the experiments for $n = 500, 1,000, 2,000, 4,000, 8,000, 16,000, \dots$. Estimate the value of a and b .

Note: The code from the slides and `stopwatch.hpp` is attached with this lab. Use optimization flag `-O2` while compiling the code, e.g., `g++ -O2 3sum.cpp -o 3sum`.

Part B: Priority queues with DELETERANDOM

Add a method `deleteRandom()` to the `MaxPQ` class that deletes and returns a random key. Your implementation should use logarithmic time in the worst case and constant space. (Hint: Exchange the item to be deleted with the item at the end of the heap, delete it, and then restore the heap invariant by swimming or sinking as necessary.)

Part C: Dynamic-median finding

Recall that the median of a set of keys is the “middle” key when the keys are sorted. If there is an even number of keys, then there are two middle keys; in this case, we define the median to be the smaller of the two middle keys. In this exercise, you will design a data type that supports the following API:

- `insert(key)`: insert a new `key` into the data structure.
- `median()`: return the median key.
- `removeMedian()`: remove and return the median key.

Your implementation should support `insert()` in logarithmic time, `median()` in constant time, and `removeMedian()` in logarithmic time.

Hint: Keep the median key in `v`; use a max-oriented heap for keys less than the key of `v`; use a min-oriented heap for keys greater than the key of `v`. To insert, add the new key into the appropriate heap, replace `v` with the key extracted from that heap.

Part D: Computational number theory

Write a program that prints out all integers of the form $a^3 + b^3$ where a and b are integers between 0 and n in sorted order, without using excessive space.

A brute-force approach would generate all n^2 pairs (a, b) , compute $a^3 + b^3$ for each, and then sort the resulting list — this takes time proportional to $n^2 \log n$ but requires space proportional to n^2 to store all the pairs. The following algorithm matches this running time while using only space proportional to n , by using a priority queue to generate and consume the sums in sorted order without ever materializing all n^2 of them at once.

- Create a priority queue that can hold n items, where each item is a tuple of the form $(i^3 + j^3, i, j)$.
- Initialize the priority queue by inserting the n items $(0^3 + 0^3, 0, 0)$, $(1^3 + 1^3, 1, 1)$, $(2^3 + 2^3, 2, 2)$, $\dots$, $(n^3 + n^3, n, n)$.
- While the priority queue is not empty, do the following:
 - Remove the smallest item $(i^3 + j^3, i, j)$ from the priority queue and print it.
 - If $j < n$, insert the item $(i^3 + (j+1)^3, i, j+1)$ into the priority queue.
 (Note: The priority queue will never contain more than n items, since for each i there is at most one item of the form $(i^3 + j^3, i, j)$ in the priority queue at any time.)

Hint: This algorithm removes the *smallest* item at each step, so you need a min-oriented priority queue (`MinPQ`) rather than the `MaxPQ` from Part B. You can build one analogously to `MaxPQ`, just replacing every `operator < comparison` with `operator >` (so `sink/swim` maintain a min-heap instead of a max-heap).

As a second part of this exercise, modify your program to find all distinct pairs of integers (a, b) and (c, d) such that $a^3 + b^3 = c^3 + d^3$. For example, $1729 = 1^3 + 12^3 = 9^3 + 10^3$ is one such number (the *Hardy–Ramanujan* number).